Robust Autonomous Vehicle Pursuit

Muhammad Umer Abbasi Qadeer Khan
umer.abbasi@tum.de qadeer.khan@tum.de

Abstract

In this work, we extend the Robust Autonomous Vehicle Pursuit without Expert Steering Labels framework to handle multi-vehicle traffic scenes. Unlike the original setup, which assumes a single target vehicle in view, our system allows a user to select a specific vehicle to follow among multiple surrounding cars. The ego vehicle uses an RGB camera to identify the chosen target and follows it reactively while maintaining a safe distance. We introduce a tracker-based pipeline that generates target masks using a tracker and combines them with RGB input for pose estimation through a convolutional neural network. To improve dataset generation efficiency, we propose a method for generating a three-degree-of-freedom car pose dataset in CARLA using only stereo image captures. For control, we reuse the multilayer perceptron trained on model predictive control labels from the original framework to predict the ego vehicle's lateral and longitudinal control commands. Experiments in the CARLA simulator show that our system achieves stable pursuit behavior in multi-vehicle environments. While the quality of the generated dataset influences the overall performance, the proposed tracking and control pipeline demonstrates effective generalization to complex traffic scenarios.

1. Introduction

Recent progress in autonomous driving has sparked growing interest in developing robust systems that can operate reliably in complex traffic scenarios. Among these, autonomous vehicle pursuit, where an ego vehicle follows a moving target vehicle, represents a challenging task that combines perception, prediction, and control under dynamic environments. Such systems can be used for cooperative driving, convoy formation, and traffic monitoring, where maintaining a safe following distance and adapting to varying traffic conditions are essential.

The original Robust Autonomous Vehicle Pursuit (RAVP) without Expert Steering Labels framework [1] demonstrated a learning-based method for pursuing a single target vehicle using a single RGB camera. The ego vehicle reactively followed the target without relying on predefined routes and achieved robust control performance across different terrains. However, the original setup was limited to scenarios with a single visible target and could not handle multi-vehicle traffic where the target of interest needs to be identified and selected.

In this work, we extend the RAVP framework to handle multi-vehicle traffic scenes. Our system enables a user to select a specific vehicle to follow among multiple surrounding cars, allowing for pursuit behavior in realistic traffic conditions. The ego vehicle uses an RGB camera to identify the chosen target and follows it reactively while maintaining a safe distance. To achieve this, we propose a tracker-based pipeline that integrates target masks from a tracker with RGB input for pose estimation through a convolutional neural network (CNN).

To improve the efficiency of data generation, we introduce a method for generating a three-degree-of-freedom (3-DoF) car pose dataset in CARLA using only stereo image captures, enabling a dataset generation approach that is not limited to the simulator. For control, we reuse the multilayer perceptron (MLP) trained on model predictive control (MPC) labels from the original RAVP framework to predict lateral and longitudinal control commands for the ego vehicle.

We extensively evaluated our approach in the CARLA simulator under various traffic conditions. Our results show that the proposed system achieves stable pursuit behavior in multi-vehicle environments. Although the performance is affected by the quality of the generated dataset, our analysis indicates that the tracking and control pipeline generalizes well to more complex and realistic driving scenarios.

In summary, the main contributions of this work are as follows:

- We extend the RAVP framework to multi-vehicle environments, enabling a user to select and pursue a specific target within traffic using only RGB input.
- We propose a tracker-based perception pipeline that fuses target masks and RGB data for robust pose estimation.
- We introduce an efficient pose dataset generation method that estimates 3-DoF car poses from stereo images in CARLA.

- We evaluate our approach on the CARLA simulator and demonstrate its ability to achieve stable pursuit despite imperfect dataset generation.

2. Background

2.1. Robust Autonomous Vehicle Pursuit without Expert Steering Labels

The original RAVP framework introduced a learning-based approach for vehicle pursuit, where an ego vehicle follows a moving target using only a single RGB camera. The framework utilizes a CNN to estimate the relative pose of the target vehicle and an MLP to predict control commands based on this estimate. A 3D object detector was used on LiDAR scans to generate ground-truth pose labels for the pose estimation CNN. An MPC was then applied to these estimated poses to produce ground-truth lateral and longitudinal control values for training an MLP, eliminating the need for expert steering labels. To improve robustness, RAVP introduced a data augmentation technique that synthesizes novel views of the target vehicle using depth-based image rendering, allowing the model to recover from off-trajectory deviations. The method demonstrated strong generalization and real-time performance in the CARLA simulator across a wide range of driving scenarios

2.2. Segment Anything Model 2

The recently introduced Segment Anything Model 2 (SAM 2) [2] provides a powerful foundation model for image and video segmentation. It extends the original SAM by incorporating a memory-based architecture that enables efficient mask propagation across frames, making it suitable for video object tracking and long-term segmentation tasks. SAM 2 can generate high-quality masks for arbitrary objects from sparse user prompts, such as clicks or boxes, while maintaining temporal consistency. In our work, we use SAM 2 during dataset generation to obtain precise instance segmentation masks of vehicles from left RGB camera images. These masks are later used to extract target vehicle poses and create ground-truth data for training our pose estimation network. The use of SAM 2 allows us to generate reliable annotations efficiently without relying on simulator-provided labels, making our dataset generation approach more flexible and scalable.

2.3. FoundationStereo

FoundationStereo (FS) [3] is a recent foundation model for stereo depth estimation that demonstrates strong zero-shot generalization across diverse visual domains. It leverages a large-scale synthetic dataset and a hybrid architecture combining convolutional and transformer-based cost aggregation to produce accurate disparity maps without domain-specific fine-tuning. In our work, we use FoundationStereo during dataset generation to estimate disparity maps from stereo RGB image pairs. The resulting disparity maps are converted into depth maps for the left camera view and later combined with SAM 2-generated masks to compute accurate 3-DoF pose ground truths for target vehicles. This allows our dataset generation pipeline to remain efficient and independent of simulator-provided labels.

2.4. Model Predictive Control

Model Predictive Control (MPC) is a well-established optimization-based control technique widely used in autonomous driving for trajectory tracking and motion planning. It predicts the future states of a system over a finite time horizon and computes optimal control inputs by minimizing a cost function subject to dynamic and physical constraints. In RAVP, MPC is employed offline to generate supervision signals for learning-based controllers. Specifically, the MPC built upon the kinematic bicycle model is used, which approximates vehicle motion using a single front and rear wheel representation while capturing essential dynamics such as steering and velocity constraints. This formulation enables accurate and computationally efficient trajectory prediction, allowing for the reliable generation of lateral and longitudinal control labels for network training without requiring expert driving data.

3. Methodology

3.1. Pose Dataset Generation

To train the pose estimation network, we require a dataset containing RGB images, instance masks and corresponding 3-DoF target vehicle poses, defined by (d_x, d_y, d_{yaw}), in the ego vehicle coordinate frame. Since simulator-provided ground truth data are not available in real-world settings, we design a self-contained pipeline to generate these labels directly from stereo RGB image sequences. The process integrates object detection, instance segmentation, depth estimation, and geometric projection.

We begin by capturing stereo RGB images from a camera set mounted on a vehicle, which roams through different towns in CARLA to record diverse traffic scenarios and environmental conditions. The left-view RGB image from each stereo pair is first processed using a YOLO object detector to identify all visible vehicles. The resulting bounding boxes are used as initialization prompts for SAM2, which produces pixel-accurate instance segmentation masks for each detected vehicle. To ensure temporal consistency, the generated masks are propagated across $N = 80$ consecutive frames, which corresponds to the maximum sequence length permitted by GPU memory constraints. This process is repeated until the all the rgb frames are covered, producing instance-level masks where each unique non-zero ID corresponds to a distinct vehicle.

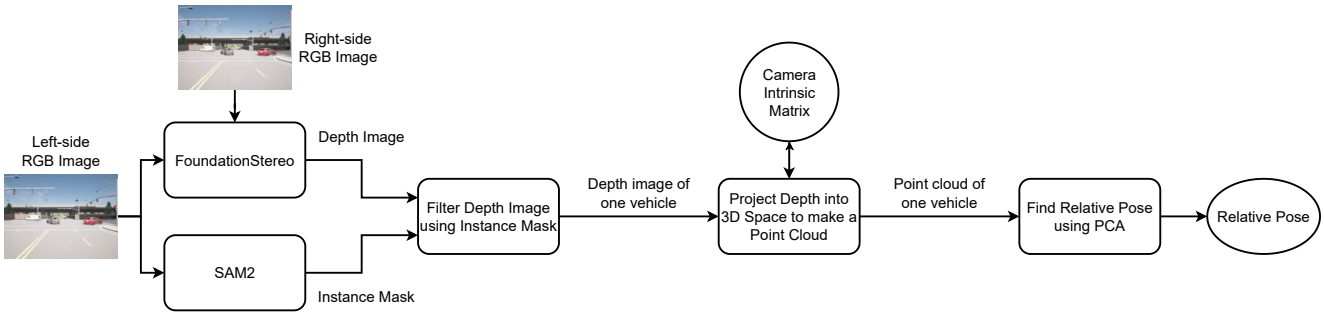

Figure 1. Overview of the proposed pipeline for pose dataset generation. Stereo RGB image pair is processed by FoundationStereo to obtain depth image for the left-side image and left-side RGB image (from stereo pair) by SAM2 for instance segmentation mask. The filtered depth mask is projected into 3D space using the camera intrinsic matrix to generate a point cloud, from which the relative pose is computed using PCA. Repeat the process for all instance ids corresponding to vehicles, in the instance mask.

For depth estimation, we employ FoundationStereo to compute disparity maps from stereo RGB pairs. Using known stereo camera parameters, including the baseline and intrinsic camera matrix, the disparity maps are converted into depth maps for the left camera view. At this stage, each left RGB frame is associated with a dense depth map and its corresponding instance segmentation mask.

For each non-zero instance ID representing a vehicle, we filter the depth map using the associated mask to isolate the depth values belonging to that object. The masked depth is then projected into 3D space using the camera intrinsics to form a point cloud representation of the vehicle. We apply Principal Component Analysis (PCA) to this point cloud to estimate its geometric structure. The mean of the point cloud provides (d_x, d_y, d_z), representing the relative position of the vehicle in the camera coordinate frame, where the CARLA convention defines $+x$ as forward, $+y$ as right, and $+z$ as up. The principal axis with the highest variance corresponds to the vehicle's longitudinal direction, which determines the yaw angle d_{yaw}.

Finally, the pose (d_x, d_y, d_{yaw}) for each detected vehicle is transformed into the ego-vehicle coordinate frame to serve as the ground-truth target pose, while d_z is omitted from the dataset as it is not required for 3-DoF control. This procedure is repeated for every frame, resulting in a comprehensive pose dataset covering all vehicles visible in the RGB sequences. The proposed pipeline, as shown in Fig. 1, enables efficient and scalable dataset generation without reliance on simulator-provided labels, making it suitable for both synthetic and real-world data. Using this pipeline, we collected 2000 frames per sequence across multiple CARLA towns. Specifically, two sequences were recorded in Town 03, two in Town 05, one in Town 02 and one in Town 04, each capturing diverse traffic conditions and layouts. The sequence from Town 04 is used for validation, while the remaining sequences are used for training.

3.2. Pose CNN Training

We train a convolutional network (PoseCNN) to regress the 3-DoF target pose (d_x, d_y, d_{yaw}) in the ego-vehicle frame from a single RGB image and a corresponding bounding box mask. Each training sample consists of an RGB frame, a binary bounding box mask, and the ground-truth pose.

The bounding box mask is derived from the instance segmentation mask of the target vehicle. This design choice allows the network to remain compatible with a lightweight tracker used during inference, which outputs bounding boxes instead of pixel-level masks. The RGB image and bounding box mask are concatenated channel-wise and used as input to the PoseCNN model.

PoseCNN follows a standard convolutional encoder followed by a regression head that outputs the pose vector (d_x, d_y, d_{yaw}). We apply mask-based data augmentations to improve generalization and robustness. Specifically, the bounding box mask is randomly scaled (expanded or shrunk) and translated within a small range to simulate localization noise that may occur during tracking. The RGB image itself is not augmented, ensuring consistent visual context.

The network is trained using a weighted L2 loss between the predicted and ground-truth pose values, where separate weights are assigned to the dx, dy, and $dyaw$ components to balance their contributions.

3.3. Inference and Control

During inference, the system operates in a closed-loop manner where the ego vehicle continuously perceives, estimates, and reacts to the motion of a selected target vehicle. The process begins with the user selecting a target vehicle by specifying its bounding box in the initial frame. A lightweight tracker then maintains the target's bounding box across subsequent frames, providing the region of interest to the perception module.

For each frame, the RGB image and the correspond-

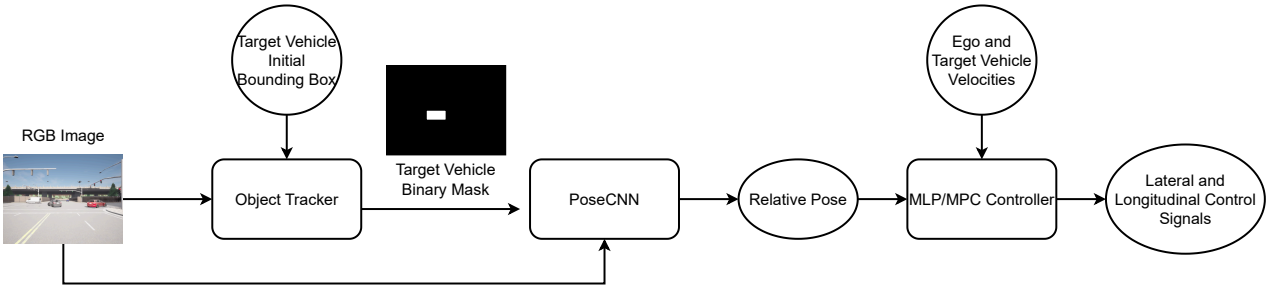

Figure 2. Block diagram of the proposed inference framework. The RGB image is processed by the tracker to detect and track target vehicles using a user provided bounding box. PoseCNN estimates the relative pose between the ego and target vehicles, using the RGB image and target vehicle mask. The relative pose, along with ego and target vehicle velocities, is provided to the MLP/MPC controller to generate lateral and longitudinal control signals for vehicle motion control.

ing bounding box mask from the tracker are passed to the trained PoseCNN. The network outputs the relative 3-DoF pose (d_x, d_y, d_{yaw}) of the target vehicle in the ego-vehicle coordinate frame. These pose estimates describe the position and orientation of the target with respect to the ego vehicle and are used to generate control commands that drive the pursuit behavior.

To control the ego vehicle, we evaluate two approaches: a learned MLP controller and a classical MPC. The MLP is taken from the original RAVP framework, which is trained on control labels generated by the MPC. Both controllers output the lateral (steering) and longitudinal (throttle) control commands required for pursuit.

This pipeline, as shown in Fig. 2, allows the ego vehicle to track and follow the chosen target reactively in real time while maintaining stability and safe separation. The integration of tracking, pose estimation, and control enables robust pursuit behavior across diverse traffic conditions.

4. Experiments and Results

4.1. PoseCNN Evaluation and Analysis

PoseCNN was trained end-to-end by minimizing the L2 loss between the ground-truth and predicted pose values (d_x, d_y, d_{yaw}). During inference experiments in the CARLA simulator, we observed that the ego vehicle successfully detected, tracked, and followed the selected target vehicle using our proposed pipeline. The ego maintained pursuit through straight and curved trajectories and was capable of following the target through turns, demonstrating that the learned pose estimation and control pipeline functioned as intended.

However, when the ego vehicle approached the target within a range of approximately 4–10 meters, its behavior became unstable. Specifically, the vehicle began oscillating laterally, making abrupt left and right steering adjustments even when the target vehicle was moving in a straight line. This caused the ego to change lanes repeatedly and exhibit non-smooth motion at close distances. Despite this issue, the overall pursuit task was still completed successfully. We tested both the MPC and MLP controllers, and both exhibited similar behavior.

To further investigate this problem, we analyzed the pose predictions from the CNN models and our dataset generation PCA outputs by comparing their values with the ground-truth (d_x, d_y, d_{yaw}) from the CARLA simulator. Figure 3 shows the predicted outputs versus their ground-truth counterparts for each pose dimension. We include results from multiple configurations: (i) the original RAVP model (denoted as Original model), (ii) our PoseCNN model (denoted as New model), (iii) PCA-derived ground truth from CARLA depth and instance masks (denoted as PCA), and (iv) PCA-derived ground truth using SAM2 masks and FoundationStereo depth maps.

From these plots, we observe that the predicted d_x and d_y values of PoseCNN closely follow the PCA-based ground truth, with variance levels similar to those of the original RAVP model. Therefore, we believe these are not the primary cause of the unstable lateral behavior. However, for d_{yaw}, both the PCA-based and PoseCNN predictions show significant scatter and deviation from the ground truth. This large yaw error likely explains the oscillatory steering behavior observed during close-range pursuit.

4.2. Evaluating VLM-based d_{yaw} Estimation

Given that our d_x and d_y errors are comparable to the original RAVP model, we focused on improving d_{yaw}. We explored a vision–language model (VLM) as an auxiliary yaw annotator, with the goal of replacing PCA-derived yaw while keeping d_x and d_y from the PCA pipeline (SAM2 masks + FoundationStereo depth). Concretely, we queried a VLM (Gemini-2.5-Flash) with the RGB frame cropped to the target vehicle's bounding box and requested the yaw angle of the vehicle relative to the ego camera. We did it three times for each frame and take the mean yaw angle as

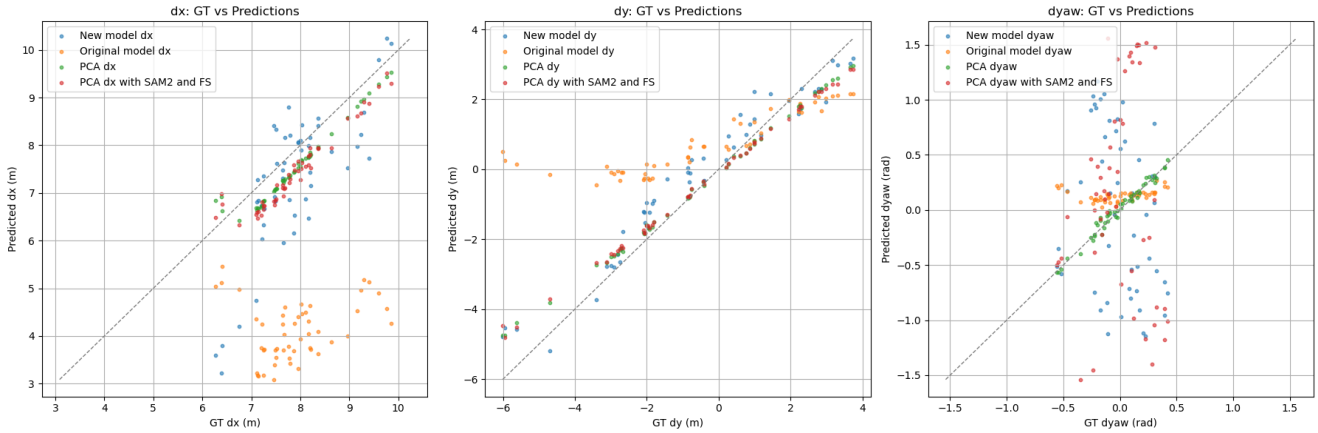

Figure 3. Comparison of predicted and ground-truth pose components (d_x, d_y, d_{yaw}) across different configurations. The plots include results from multiple configurations: (i) the original RAVP model (denoted as Original model), (ii) our PoseCNN model (denoted as New model), (iii) PCA-derived ground truth from CARLA depth and instance masks (denoted as PCA), and (iv) PCA-derived ground truth using SAM2 masks and FoundationStereo depth maps.

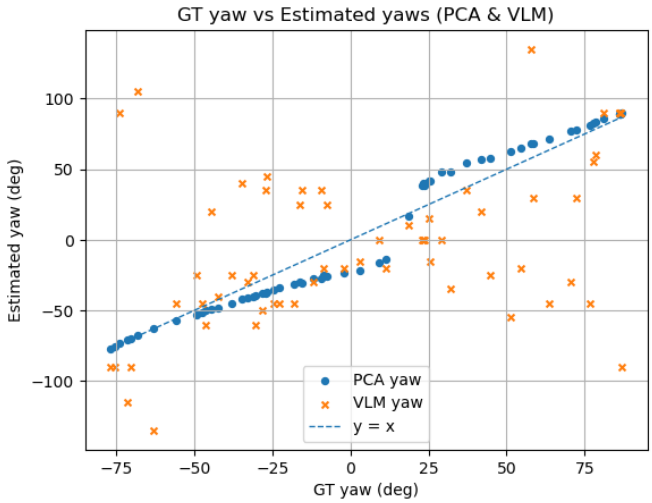

Figure 4. VLM-predicted d_{yaw} and PCA-based d_{yaw} against the CARLA ground truth.

d_{yaw}. The returned yaw (in degrees) was compared against the CARLA ground truth.

Figure 4 plots VLM-predicted d_{yaw} and PCA-based d_{yaw} against the CARLA ground truth. We observe that PCA-based yaw is especially poor in the range of $-25°$ to $+25°$, which is the most critical region for stable pursuit. The VLM yaw is generally unreliable across the entire range, with predictions scattered and weakly correlated with ground truth. Given these results, we do not adopt VLM-generated yaw in our final pipeline.

4.3. Target Vehicle Pursuit Results

To quantitatively evaluate the pursuit behavior, we compute four metrics following the original RAVP evaluation protocol: Route Completion (RC), Infraction Coefficient (IC), Mean Translation Error (MTE_m), and Control Difference (CD).

Route Completion (RC) measures how much of the planned episode the ego vehicle successfully completes before losing the target from sight. It is defined as the ratio between executed and total frames, i.e., $\text{RC} = N_{\text{executed}}/N_{\text{planned}}$, with higher values indicating longer uninterrupted tracking.

Infraction Coefficient (IC) penalizes lane invasions and collisions relative to the target vehicle. Each lane invasion and collision apply exponential penalties of 0.8 and 0.5 respectively:

$$\text{IC} = (0.8)^{n_{\text{invasion}}}(0.5)^{n_{\text{collision}}}.$$

A higher IC thus represents safer pursuit behavior with fewer infractions.

Mean Translation Error (MTE$_m$) quantifies spatial tracking accuracy. We compute the mean SE(2) translation error between the ego and target trajectories after temporal alignment using the Hungarian assignment algorithm. Lower MTE$_m$ implies that the ego trajectory stays spatially closer to the target path.

Control Difference (CD) measures the mean L2 distance between the ego's and target's control inputs (throttle and steering) for matched trajectory indices:

$$\mathrm{CD} = \frac{1}{N} \sum_i \|u^i_{\text{ego}} - u^i_{\text{target}}\|_2.$$

A smaller CD indicates more consistent control actions and smoother mimicry of the target's behavior.

We evaluated closed-loop pursuit in two towns, town 04 and town 05, using the same pipeline and controllers. We spawned two vehicles ahead of ego, mark the target using an bounding box, and let the ego follow the target vehicle. Please find the videos in the demo directory of the repositry. For town 04, blue car is the target and for town 05, black car is the target. Table 1 summarizes the metrics. In both cases the ego completed the full episode (RC $= 1.0$). However, the infraction coefficient (IC, higher is better) is close to zero in both runs, indicating the ego accrued lane/collision events relative to the target despite finishing the task. Spatial tracking accuracy (MTE$_m$) is town-dependent: Town-04 stays near 1.03 m on average, while Town-05 degrades to 1.69 m, consistent with the observed lateral oscillations at close range. Control mismatch (CD, lower is better) is comparable across towns (roughly $0.87{-}1.02$).

Scenario	RC (↑)	IC (↑)	MTE$_m$ (↓) [m]	CD (↓)
Town-04	1.0	0.0	1.027	1.017
Town-05	1.0	0.0	1.695	0.875

Table 1. Closed-loop pursuit metrics across towns.

Overall, pursuit succeeds end-to-end, but low IC and higher MTE$_m$ in Town-05 expose safety and stability gaps that correlate with the large d_{yaw} errors at short range. Tightening yaw estimation (e.g., temporal smoothing or geometric priors) is likely to yield the largest stability gains.

5. Conclusion

We evaluate the proposed multi-vehicle RAVP extension in the CARLA simulator across diverse towns and traffic densities, focusing on (i) pose estimation quality, (ii) closed-loop pursuit performance with both MPC and MLP controllers, (iii) robustness and generalization, and (iv) runtime.

5.1. Pose estimation quality

Two consistent trends emerge:

- d_x **and** d_y: Our PoseCNN closely tracks the PCA labels and exhibits variance comparable to the original RAVP baseline, indicating that range and lateral offset are reliably estimated across scenes.
- d_{yaw}: Both PCA-derived yaw and PoseCNN-predicted yaw show noticeably higher scatter relative to CARLA ground truth, with errors amplified at short ranges. This systematic weakness in yaw labeling/learning is the primary reason for lateral instability.

5.2. Closed-loop pursuit performance

In closed-loop simulation the ego robustly detects, tracks, and follows the user-selected target through straights and turns. However, when approaching within $\sim$4–10 m, we observe lateral oscillations (left–right steering flips) even when the target proceeds straight, increasing lane-change counts and reducing time-in-lane. Both MLP and MPC controllers exhibit similar behavior, suggesting the root cause lies in perception (yaw noise) rather than the control law. Consistent with the pose analysis, stable pursuit is maintained at medium ranges, while close-range regulation is limited by d_{yaw} noise.

5.3. Ablation: source of yaw supervision

To isolate d_{yaw} supervision quality, we evaluated a VLM-based annotator by cropping the target bounding box and querying the relative yaw. We found that VLM d_{yaw} is broadly unreliable across the full range for Gemini-2.5-flash model; consequently we do not adopt VLM-derived d_{yaw} in the final pipeline. However, we attribute the d_{yaw} error primarily to depth quality rather than masking: SAM2 yields consistently reliable masks, and we expect that replacing the current depth with a stronger model would markedly improve the PCA-derived d_{yaw}.

5.4. Generalization and robustness

Across multiple towns and traffic compositions, pursuit remains stable when the target remains visible and moderately separated; performance degrades primarily under near-field interactions where yaw noise translates into steering oscillations. The tracker-compatible design of PoseCNN (RGB+box-mask input) preserves robustness when only bounding boxes are available at test time.

5.5. Runtime

The end-to-end inference loop (tracking $\rightarrow$ PoseCNN $\rightarrow$ control) runs in real time on a single GPU in CARLA. Dataset generation is efficient and simulator-agnostic by combining SAM2 for masks and FS for disparity, avoiding reliance on simulator labels while yielding dense supervision for (d_x, d_y) and if noisy, d_{yaw}.

Takeaways. Our multi-vehicle RAVP system achieves reliable pursuit in realistic traffic with only RGB input and a user-selected target. The principal limitation is yaw estimation and reducing d_{yaw} noise at close range should directly attenuate the observed lateral oscillations.

References

[1] J Pan, C Zhou, M Gladkova, Q Khan, and D Cremers. Robust autonomous vehicle pursuit without expert steering labels. *IEEE Robotics and Automation Letters (RA-L)*, 2023. 1

[2] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 2

[3] Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. Foundationstereo: Zero-shot stereo matching. *arXiv*, 2025. 2